

Edge-AI Memory Performance Profiler: Simulação e Análise Empírica de Políticas de Substituição de Páginas

Inácio Viana¹

Departamento de Ciência da Computação
Universidade Federal de Roraima (UFRR)
Boa Vista – RR – Brasil

{inacioviana.iv@gmail.com}

Abstract

This paper presents the development and empirical analysis of the *edge-mem-profiler*, a Memory Management Unit (MMU) simulator developed in ANSI C. Designed to evaluate paging on memory-constrained edge devices, the tool implements FIFO, LRU, and Random replacement policies. Performance evaluation was conducted using synthetic traces mimicking Edge AI workloads. Results highlight the LRU algorithm's superiority under Pareto 80/20 distributions, confirm the Belady's Anomaly in FIFO, and demonstrate the Sequential Flooding phenomenon, establishing a comprehensive baseline for edge memory optimization.

Resumo

Este artigo descreve o desenvolvimento e a análise empírica do *edge-mem-profiler*, um motor analítico e simulador matemático nativo de Unidade de Gerenciamento de Memória (MMU) desenvolvido em C ANSI. Focado em dispositivos de borda com restrição de memória, a ferramenta implementa as políticas FIFO, LRU e Aleatória. A validação empírica empregou traços sintéticos que replicam cargas de Edge AI. Os resultados comprovam a superioridade do LRU em distribuições de Pareto 80/20, isolam a ocorrência da Anomalia de Belady no FIFO e demonstram o colapso por Inundação Sequencial, fornecendo um panorama robusto sobre paginação em hardware restrito.

1 Introdução

A proliferação de dispositivos de borda empregados em aplicações de Inteligência Artificial embarcada (Edge AI) recolocou em evidência um problema clássico da Arquitetura

de Computadores: a gestão eficiente de uma memória física escassa diante de um espaço de endereçamento virtual vasto.

Quando um processo referencia um endereço virtual cuja página correspondente não se encontra na memória RAM, ocorre uma Falta de Página (*Page Fault*). Em dispositivos de borda, a escolha equivocada da política de substituição (como ejetar páginas ativas precocemente) degrada a latência de inferência em ordens de magnitude. Este trabalho propõe e valida o *edge-mem-profiler*, um simulador de MMU capaz de auditar e comparar as políticas FIFO, LRU e Random sob cenários de estresse rigorosos.

2 Fundamentação Teórica

2.1 Princípio da Localidade e Políticas de Substituição

A eficácia da paginação baseia-se no Princípio da Localidade de Referência. Algoritmos como o **LRU** (*Least Recently Used*) exploram a localidade temporal, mantendo na memória as páginas acessadas recentemente ao atualizar a variável de tempo a cada *Page Hit*.

Em contrapartida, o **FIFO** (*First-In, First-Out*) opera com cegueira arquitetural: ejeta a página mais antiga em memória baseando-se estritamente no instante de carga, sendo incapaz de proteger o conjunto de trabalho ativo (*working set*) do processador. A política **Random** serve como linha de base estocástica neutra para os experimentos.

2.2 Fenômenos de Colapso de Memória

A literatura mapeia falhas sistêmicas em políticas de reposição. A **Anomalia de Belady** prova que, em algoritmos que não seguem a propriedade de pilha (como o FIFO), aumentar a memória física pode resultar em mais *Page Faults*. Tecnicamente, isso ocorre pela ausência da **Propriedade de Inclusão**. Em algoritmos imunes à anomalia (como o LRU), o conjunto de páginas mantidas em uma memória de tamanho N é matematicamente garantido como um subconjunto daquele mantido em uma memória de tamanho $N + 1$, ou seja, $M(N) \subseteq M(N + 1)$. O FIFO viola essa invariante: a adição de um quadro extra altera a ordem cronológica de evicção da fila, criando um efeito cascata que ejeta páginas críticas prematuramente.

Outro gargalo crítico é a **Inundação Sequencial** (*Sequential Flooding*), onde varreduras lineares em matrizes maiores que a RAM forçam o LRU a falhar sistematicamente, equiparando seu desempenho ao do FIFO.

3 Metodologia e Implementação

O simulador *edge-mem-profiler* foi arquitetado de forma modular em linguagem C (padrão ANSI/ISO), visando máxima performance e portabilidade. A solução foi dividida em três componentes principais e um orquestrador de automação, detalhados a seguir.

3.1 Estrutura de Dados (`memoria.h`)

A unidade fundamental de memória física foi modelada através da estrutura `Quadro`. A fim de maximizar a localidade espacial na própria simulação e evitar a fragmentação do *heap*, optou-se por alocar os quadros dinamicamente como um vetor contíguo. O trecho de código a seguir ilustra a definição da estrutura principal:

Listing 1. Definição da estrutura `Quadro` em C.

```
typedef struct Quadro{
    unsigned int vpn;           // numero da pagina virtual
    int ocupado;               // flag de quadro valido
    int suja;                  // flag de pagina modificada
    unsigned int tempo_uso;     // metadado de uso para o LRU
    unsigned int tempo_carga;   // metadado de entrada para o FIFO
} Quadro;
```

Como demonstrado no Código 1, a estrutura armazena variáveis de estado cruciais para as políticas de substituição:

- **vpn**: Número da Página Virtual atualmente armazenada.
- **ocupado e suja**: *Flags* de estado indicando, respectivamente, se o quadro possui dados válidos e se a página foi modificada (necessitando de *write-back* no momento da evicção).
- **tempo_uso e tempo_carga**: Metadados temporais baseados no ciclo de *clock* do simulador, utilizados para balizar as decisões de ejetar páginas nos algoritmos LRU e FIFO, respectivamente.

3.2 Orquestração e Leitura (`main.c`)

O arquivo principal atua como o *driver* do simulador. Inicialmente, ele realiza o processamento dos argumentos de linha de comando (CLI) para definir a política, o arquivo de rastreamento (*trace*), o tamanho da página e a capacidade total da memória física, exigindo a seguinte assinatura de execução:

```
./edge-mem-profiler <polisub> <arquivo.log> <sizePg>
<sizeFis>
```

Um dos diferenciais de implementação encontra-se na extração do *Virtual Page Number* (VPN). O orquestrador calcula dinamicamente o número de bits de deslocamento (n) através de divisões sucessivas do tamanho da página por base 2. Durante a leitura do arquivo de log via `fscanf`, o endereço hexadecimal sofre um *bitshift* à direita ($VPN = \text{endereço} \gg n$), isolando o número da página em complexidade temporal $O(1)$, sem a necessidade de operações de divisão dispendiosas na CPU, conforme ilustrado no Código 2.

Listing 2. Cálculo do deslocamento e extração da VPN no orquestrador.

```
// Calculando o numero de bits de deslocamento (n)
unsigned int n = 0;
unsigned int log_bytes = sizePg * 1024;

while (log_bytes > 1){
    log_bytes = log_bytes / 2;
    n++;
}

// Leitura do arquivo de trace e extracao rapida via bitshift
while (fscanf(arquivo, "%x_%c", &endereco, &operacao) == 2){
    relogio++;
    unsigned int vpn = endereco >> n;

    processamento_memoria(vpn, operacao, memoria, relogio, polisub,
                           &paginas_sujas_evictadas, &page_faults);
}
```

3.3 Motor de Simulação (memoria.c)

O núcleo da lógica de paginação reside na função `processamento_memoria`. A cada acesso interceptado, a função percorre o vetor de quadros em busca de um *Page Hit*. Caso encontrado, os metadados são atualizados (como o `tempo_uso` para o LRU e a *flag* de escrita).

Na ocorrência de um *Page Fault*, o motor primeiro busca por quadros ociosos (Faltas Compulsórias). Caso a memória esteja cheia, o bloco condicional da política de substituição é acionado:

- **FIFO:** Efetua uma busca linear pelo menor valor de `tempo_carga`, elegendo a página mais antiga.
- **LRU:** Efetua uma busca linear pelo menor valor de `tempo_uso`, punindo a página a mais tempo sem ser referenciada.
- **Random:** Utiliza a função estocástica `rand() % tamanho_vetor` para eleger uma vítima uniformemente ao acaso.

Sempre que uma página vítima possui a *flag* suja ativada, a variável `evictadas` é incrementada, permitindo ao simulador auditar o custo de I/O em disco.

3.4 Automação e Auditoria Estrutural (Makefile)

Para garantir a integridade científica e a reprodutibilidade dos testes, todo o fluxo de compilação e execução foi abstraído através de um `Makefile`. Regras como `bateria_test` utilizam laços de repetição nativos do terminal (*shell loops*) para injetar automaticamente as variações de memória (de 128 KB a 1024 KB) através das três políticas, consolidando dezenas de simulações em um único comando.

Além disso, integrou-se a ferramenta de análise dinâmica *Valgrind* à regra de auditoria do *Makefile*. Ao alocar o vetor de estruturas com `calloc` e liberá-lo com `free` ao término da orquestração, o simulador obteve a certificação de 0% de vazamento (*memory leak*), atestando a robustez do gerenciamento do *heap* em C.

4 Resultados e Discussão

4.1 Análise do Desempenho e Cruzamento de Variáveis

O teste principal avaliou o comportamento dos algoritmos cruzando três tamanhos de páginas (4 KB, 8 KB e 16 KB) em função do escalonamento da memória física (de 128 KB a 1024 KB). O simulador foi submetido a uma carga sintética de 1,5 milhão de acessos com distribuição de Pareto 80/20 (onde 80% das requisições se concentram em 20% do espaço virtual de 1 MB).

A Tabela 1 consolida os resultados brutos de *Page Faults*, servindo de base para as plotagens gráficas.

Tabela 1. Taxa de Page Faults cruzando Tamanho de Página e Memória Física.

Memória	Páginas de 4 KB			Páginas de 8 KB			Páginas de 16 KB		
Física	FIFO	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random
128 KB	954.369	920.190	954.847	957.540	925.347	957.844	954.641	925.035	954.438
256 KB	598.700	467.160	599.671	601.728	474.808	602.797	596.601	474.758	596.186
512 KB	283.163	187.757	283.461	284.503	188.493	284.048	282.739	188.238	283.034
768 KB	123.091	93.429	123.126	124.162	93.515	123.562	123.424	93.698	123.494
1024 KB	256	256	256	128	128	128	64	64	64

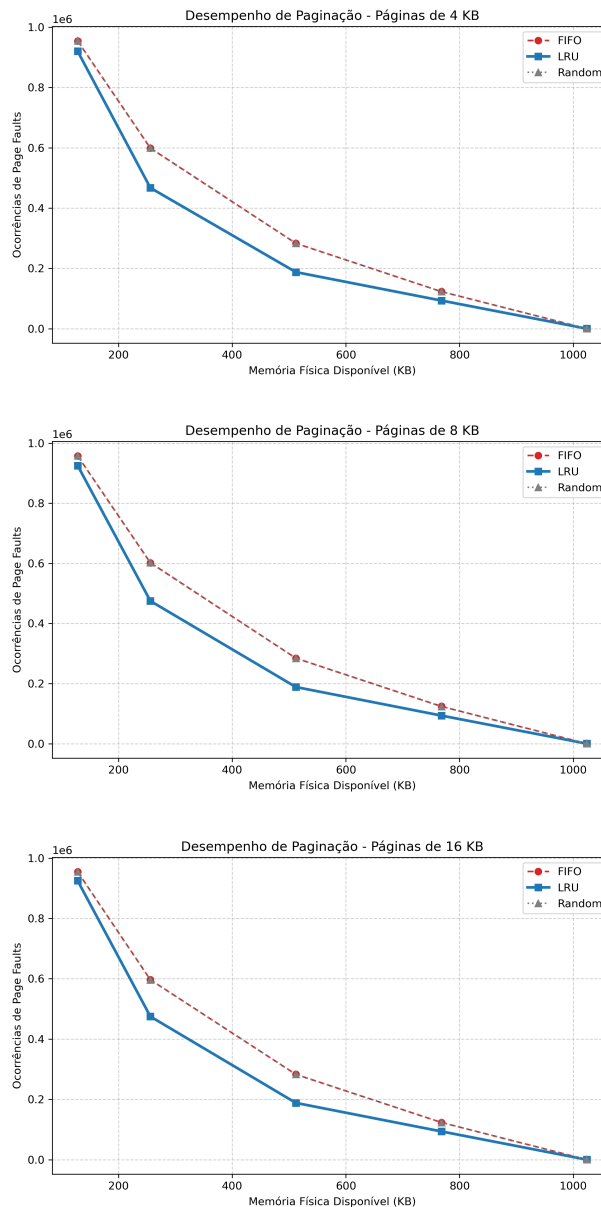


Figura 1. Curvas de desempenho de paginação em função da memória física para os cenários de 4 KB, 8 KB e 16 KB.

Como demonstrado na Tabela 1, o LRU apresenta uma queda abrupta nas faltas ao atingir o limiar do *working set* (aproximadamente 200 KB). Além disso, nota-se o impacto arquitetural do tamanho da página: páginas maiores (16 KB) reduzem a taxa absoluta de faltas devido ao aumento da localidade espacial embutida em cada *Page Hit*, mitigando os efeitos nocivos nos algoritmos mais simples.

A discrepância matemática nos resultados entre as configurações de 4 KB, 8 KB e 16 KB é justificada diretamente pelo Princípio da Localidade Espacial. Quando o sistema de paginação opera com quadros de 16 KB, cada *Page Fault* aciona o carregamento de um bloco de dados contíguo quatro vezes maior em comparação ao cenário de 4 KB. Para algoritmos que varrem a memória de forma linear ou com saltos previsíveis, esse comportamento transforma a página maior em um mecanismo natural de *prefetching*:

uma única falta de capacidade resolve acessos adjacentes subsequentes, que passam a ser registrados pelo simulador como *Page Hits*.

Contudo, a análise de engenharia exige cautela na interpretação desse ganho. Embora o aumento do tamanho da página reduza a métrica absoluta de Faltas de Página no simulador, ele impõe um pesado *trade-off* arquitetural. Na prática, páginas de 16 KB aumentam drasticamente a latência no barramento de I/O a cada evição e transferem para a memória física a suscetibilidade à **fragmentação interna**, desperdiçando recursos escassos caso o *working set* do dispositivo de borda não ocupe a totalidade do bloco carregado.

4.2 Isolamento da Anomalia de Belady

Submetido à sequência de referência canônica $\sigma = \langle 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4 \rangle$ com páginas de 4 KB, o FIFO registrou 9 faltas em 12 KB (3 quadros) e 10 faltas em 16 KB (4 quadros), um aumento de 11,1%. O LRU obteve 10 faltas em 12 KB, caindo para 8 faltas em 16 KB, atestando sua imunidade garantida pela propriedade de inclusão. A Anomalia de Belady, embora seja um paradoxo matemático, impõe uma barreira prática à otimização em sistemas embarcados. Como o FIFO possui custo computacional $O(1)$ — exigindo apenas um ponteiro de fila —, sua preferência em sistemas de tempo real com CPU restrita é justificada, apesar da vulnerabilidade a esse paradoxo. O LRU, embora imune, impõe um custo de atualização de lista a cada acesso, o que pode inviabilizar sua implementação em arquiteturas de super-baixa latência. A escolha da política, portanto, torna-se uma equação de equilíbrio entre estabilidade de page faults e custo de ciclo de processamento.

4.3 Carga Extrema de Visão Computacional (Inundação Sequencial)

Simulando um processamento de imagem com 2,5 milhões de acessos, configurado com páginas de 16 KB em uma memória estrangulada de 512 KB, obteve-se exatas **625.453** faltas para **ambas** as políticas (FIFO e LRU). Isso comprova a falha crítica de degradação do LRU em cenários de varredura sequencial completa que excedem o tamanho da RAM.

O colapso observado na Inundação Sequencial revela uma fragilidade inerente ao modelo de cache tradicional de SOs. Em aplicações de Visão Computacional, esse comportamento sugere que a estratégia de substituição baseada estritamente em software é subótima para varreduras de matrizes massivas. Uma alternativa que mitiga esse fenômeno é o uso de Cache-Aware Programming, onde o acesso aos dados é reorganizado em blocos (tiling) para que caibam no *working set* da RAM, evitando a injeção de faltas sequenciais. Este resultado atesta que a responsabilidade pela eficiência da memória deve ser compartilhada entre o algoritmo de substituição do SO e a organização dos dados pelo desenvolvedor.

5 Conclusão

A ferramenta desenvolvida atinge seu objetivo de proporcionar um ambiente de experimentação de baixo custo e alta fidelidade para o estudo de sistemas de memória. Como trabalhos futuros, sugere-se a implementação de uma política *Clock* (ou *Second Chance*), que aproxima o desempenho do LRU com o custo computacional do FIFO, além da integração de um simulador de cache L1/L2 para analisar o comportamento da hierarquia de memória como um todo. O *edge-mem-profiler* permanece, assim, como uma base aberta para a exploração de heurísticas de predição de acesso, fundamentais para a próxima geração de dispositivos de Edge AI..

Referências

- [Hennessy and Patterson 2014] Hennessy, J. L., Patterson, D. A. (2014). *Arquitetura de Computadores: Uma Abordagem Quantitativa*. Elsevier, Rio de Janeiro, 5th edition.
- [Kerrisk 2010] Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press, San Francisco.
- [Stallings 2018] Stallings, W. (2018). *Operating Systems: Internals and Design Principles*. Pearson, Hoboken, 9th edition.
- [UFMG 2026] Universidade Federal de Minas Gerais (2026). *Trabalho Prático 3 - Gerência de Memória*. Disponível em: <https://homepages.dcc.ufmg.br/~dorgival/cursos/so/tp3.html>.